Capítulo 3: Ramificaciones en Git

Tutorial de Git basado en Pro Git (2.ª ed.) · Chacon y Straub · Elaborado por Ricardo · UNIAGRARIA

Introducción

Las ramas permiten divergir de la línea principal de desarrollo y trabajar sin afectarla. En Git son extremadamente ligeras, y por eso se usan de forma constante.

3.1 ¿Qué es una rama?

Cada commit guarda un puntero al árbol de archivos de la instantánea y a su(s) commit(s) padre(s). Una rama es simplemente un puntero móvil a uno de esos commits. La rama por defecto se llama main (históricamente master) y HEAD es un puntero especial que indica la rama o el commit en el que se está trabajando.

```
git branch prueba        # crea la rama (no cambia a ella)
git log --oneline --decorate   # muestra a dónde apunta cada rama y HEAD
git switch prueba         # cambia de rama
git checkout prueba       # forma clásica equivalente
```

3.2 Procedimientos Básicos para Ramificar y Fusionar

```
git switch -c iss53          # crea y cambia (git checkout -b iss53)
# ... trabajar y confirmar ...
git switch main
git merge iss53
git branch -d iss53        # borra la rama ya fusionada
```

- Fast-forward: si la rama a fusionar está directamente adelante de la actual, Git solo mueve el puntero.
- Fusión a tres bandas (three-way): si las historias divergieron, Git usa el ancestro común y los dos extremos para crear un commit de fusión con dos padres.

Conflictos

Si las dos ramas cambiaron la misma parte de un archivo, Git detiene la fusión y marca el archivo:

```
<<<<<<< HEAD
versión de la rama actual
=======
versión de la rama que se fusiona
>>>>>>> iss53
```

Se edita el archivo para dejar el contenido final, se prepara con git add y se termina con git commit. git status indica qué archivos están sin fusionar (unmerged).

3.3 Gestión de Ramas

```
git branch               # lista ramas (* marca la actual)
git branch -v            # último commit de cada rama
git branch --merged      # ramas ya fusionadas en la actual
git branch --no-merged   # ramas con trabajo sin fusionar
git branch -d rama       # borra si está fusionada
```

```
git branch -D rama        # fuerza el borrado
git branch -m viejo nuevo # renombra
```

3.4 Flujos de Trabajo Ramificados

- Ramas de largo recorrido: por ejemplo main (código estable), develop (integración) y ramas de pruebas, que se fusionan hacia las más estables.
- Ramas puntuales (topic branches): ramas cortas para una sola funcionalidad o corrección, que se crean, se fusionan y se borran.

3.5 Ramas Remotas

Las ramas de seguimiento remoto (origin/main) son punteros locales que reflejan el estado de las ramas del servidor; se actualizan con git fetch y no se pueden mover a mano.

```
git push origin mi-rama
git push -u origin mi-rama         # -u deja configurado el seguimiento
git switch mi-rama                 # crea una rama local que sigue a origin/mi-
rama
git branch -u origin/mi-rama       # configurar seguimiento
git branch -vv                     # ver el seguimiento
git push origin --delete mi-rama   # eliminar rama remota
```

3.6 Reorganizar el Trabajo Realizado (rebase)

git rebase toma los cambios confirmados en una rama y los vuelve a aplicar sobre otra, produciendo una historia lineal.

```
git switch experimento
git rebase main
git switch main
git merge experimento      # ahora es fast-forward
```

> **Nota:** Regla de oro: no hagas rebase de commits que ya hayas publicado en un repositorio compartido, porque reescribe la historia y causa problemas a quienes ya la descargaron.

Resumen

Una rama es un puntero ligero a un commit. Crear, cambiar y fusionar ramas es barato y permite trabajar en paralelo con flujos de largo recorrido o ramas puntuales. Los remotos se siguen con ramas origin/*, y el rebase ofrece una historia lineal siempre que no se aplique sobre commits ya publicados.

Referencia

Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. https://git-scm.com/book/es/v2 — Capítulo 3.